

Proiect Structuri de Date Avansate

Sistem de Gestionare a Sarcinilor

Flavius Fetitciu

Informatica Romana, Anul II

Mai 2026

Cuprins

1	Tematica proiectului	1
2	Descrierea structurilor alese și arhitectura proiectului	1
3	Explicația algoritmilor implementați	2
3.1	Algoritmi pentru Min-Heap	2
3.2	Algoritmi pentru Hash Table	2
4	Analiza complexității	2
4.1	Complexitatea în Timp	3
4.2	Complexitatea în Spațiu	3
5	Exemple de rulare	3
6	Dificultăți întâmpinate	4

1 Tematica proiectului

Proiectul implementează un Sistem de Gestionare a Sarcinilor. Într-un scenariu practic, cum ar fi gestiunea activităților zilnice ale unui utilizator sau planificarea proceselor într-un sistem de operare, sarcinile trebuie procesate strict în funcție de gradul lor de urgență. Totodată, este necesară interogarea rapidă a detaliilor unei sarcini specifice pe baza unui identificator unic ID, fără a perturba ordinea de execuție.

2 Descrierea structurilor alese și arhitectura proiectului

Pentru a îndeplini cerințele operaționale la un nivel optim de performanță, arhitectura aplicației se bazează pe două structuri de date fundamentale, implementate manual de la zero și sincronizate:

- **Grămadă Binară de Minim (Min-Heap):** Este utilizată pentru stocarea și extragerea sarcinilor în ordinea priorității. Min-Heap-ul menține întotdeauna sarcina cu prioritatea cea mai mică în poziția de rădăcină. Este implementat pe baza unui vector alocat dinamic.
- **Tabelă de Dispersie (Hash Table) cu Înlănțuire:** Este utilizată pentru a asigura căutarea instantanee a oricărei sarcini folosind ID-ul generat automat. Structura constă dintr-un tablou de pointeri către liste simplu înlănțuite, capabile să gestioneze coliziunile.

Arhitectura generală: Cele două structuri funcționează în tandem. La adăugarea unei sarcini, aceasta primește un ID unic și este inserată în ambele structuri. La extragerea celei mai urgente sarcini din Heap, sistemul preia ID-ul acesteia și o elimină instantaneu și din Tabela de Dispersie. Datele sunt persistate în fișierul `tasks.txt` la fiecare modificare.

3 Explicația algoritmilor implementați

Aplicația utilizează o serie de algoritmi specifici pentru menținerea proprietăților structurilor de date:

3.1 Algoritmi pentru Min-Heap

- **Inserarea:** O sarcină nouă este adăugată la sfârșitul vectorului. Algoritmul compară prioritatea nodului nou cu cea a părintelui său. Dacă nodul nou are o prioritate mai mică, acesta este interschimbat cu părintele. Procesul se repetă iterativ până când proprietatea de Min-Heap este restabilită.
- **Extragerea:** Sarcina din rădăcină este extrasă și înlocuită cu ultimul element din vector. Algoritmul parcurge apoi arborele în jos, comparând noul nod rădăcină cu copiii săi și interschimbându-l cu cel mai mic dintre aceștia, repetând procesul până la restabilirea echilibrului.

3.2 Algoritmi pentru Hash Table

- **Funcția Hash:** Utilizează algoritmul modulo pentru a transforma ID-ul unic într-un index valid din tablou.
- **Gestiunea Coliziunilor:** În caz de coliziune, noul nod este adăugat la începutul listei înlănțuite corespunzătoare indexului. Algoritmul de ștergere parcurge lista utilizând doi pointeri pentru a izola și șterge nodul vizat fără a rupe restul lanțului.

4 Analiza complexității

Eficiența proiectului se bazează pe utilizarea optimă a celor două structuri, garantând timpi de execuție rapizi. Considerăm N numărul de sarcini din sistem.

4.1 Complexitatea în Timp

- **Inserarea unei sarcini:** Inserarea în Heap necesită $\mathcal{O}(\log N)$, iar în Hash Table necesită în medie $\mathcal{O}(1)$. Timpul total de inserare este $\mathcal{O}(\log N)$.
- **Extragerea celei mai urgente sarcini:** Eliminarea din rădăcina Heap-ului și reechilibrarea necesită $\mathcal{O}(\log N)$. Găsirea și ștergerea ID-ului din Hash Table necesită, în medie, $\mathcal{O}(1)$. Timp total: $\mathcal{O}(\log N)$.
- **Căutarea după ID:** Datorită Tabelei de Dispersie, localizarea unei sarcini se face în $\mathcal{O}(1)$ în caz mediu.

4.2 Complexitatea în Spațiu

Memoria utilizată scalează liniar cu numărul de elemente. Heap-ul folosește un vector de dimensiune N , iar Hash Table-ul utilizează N noduri alocate dinamic. Astfel, complexitatea în spațiu este $\mathcal{O}(N)$.

5 Exemple de rulare

Aceste exemple demonstrează funcționarea sistemului pentru gestiunea unor sarcini zilnice uzuale.

1. Adăugarea sarcinilor în sistem. Prioritatea 1 reprezintă urgența maximă.

```
Choice: 1
Enter title: Invatare_Examen
Enter priority: 1
Task added with ID: 1
```

```
Choice: 1
Enter title: Cumparaturi
Enter priority: 3
Task added with ID: 2
```

```
Choice: 1
Enter title: Proiect_SDA
Enter priority: 2
Task added with ID: 3
```

2. Vizualizarea sarcinilor și a organizării lor

```
Choice: 4
ID: 1, Title: Invatare_Examen, Priority: 1
ID: 2, Title: Cumparaturi, Priority: 3
ID: 3, Title: Proiect_SDA, Priority: 2
```

```
Choice: 5
Bucket 1: [1: Invatare_Examen] -> NULL
Bucket 2: [2: Cumparaturi] -> NULL
Bucket 3: [3: Proiect_SDA] -> NULL
```

3. Căutarea rapidă și extragerea sarcinii prioritare

```
Choice: 3
Enter ID to search: 3
Found Task - Title: Proiect_SDA, Priority: 2

Choice: 2
Executed Task - ID: 1, Title: Invatare_Examen, Priority: 1
```

6 Dificultăți întâmpinate

- **Sincronizarea structurilor de date:** Principala provocare a fost menținerea coeziunii între Min-Heap și Hash Table. A fost necesară proiectarea atentă a logicii de extragere astfel încât, în momentul în care elementul minim părăsește Heap-ul, ID-ul său să fie transmis corect către metoda de ștergere a tabelii de dispersie.
- **Citirea corectă a șirurilor de caractere:** Implementarea inițială bloca citirea sarcinilor care conțineau spații. Soluționarea a presupus curățarea buffer-ului și utilizarea funcției getline pentru citirea completă. Această modificare a necesitat și refactorizarea formatului de salvare și încărcare din fișier.
- **Gestiunea memoriei:** Tabela de dispersie implică zeci de pointeri alocați dinamic. Scrierea destructorului clasei a fost o procedură meticuloasă, asigurându-mă că fiecare nod din listele înlănțuite este eliberat la terminarea programului.